בס״ד

① 212319131 ליאור גולר

פתרון בחינה 2023ב א׳

שאלה 1.

א. כן, הפרוטוקול המוצע אכן נותן מניעה הדדית.
נשים לב כי כל תהליך שרוצה להיכנס לקטע הקריטי, צריך לעבור בלולאת for
שעוברת על כל התהליכים הקודמים, ועבור כל אחד מהם תהליך הרוצה להיכנס
לקטע הקריטי, צריך לעבור כ 2 לולאות while.
תהליך הרוצה להיכנס - אם יש תהליך אחר שבאמצע "שלבי הכניסה" התהליך
ממתין.
עכשיו, אם המספר של j שווה ל0 - זה אומר שהוא כלל לא רוצה להיכנס לקטע
הקריטי או שכבר יצא ממנו כבר, ולכן לא צריך להתייחס אליו משום שאינו
לא ממתין אותם, ולכן עוברים לתהליך הבא ב for.
אם המספר שלו שונה מ 0, זה אומר שגם תהליך j או שהוא רוצה להיכנס גם
או שהוא כבר נמצא בקטע הקריטי - במקרה זה בודקים את התנאי השני
של לולאת ה- while,
תנאי זה אומר, שאם המספר של תהליך j שווה בדיוק כרגע הוא קטן מאשר
~~[illegible] (המזהה שלו) לפי [illegible]~~
המספר של התהליך i, אז i צריך לחכות לו, כי הוא קיבל מספר לפני.
אם לשניהם יש אותו מספר, דבר זה ייתכן משום שקבלת המספר הוא לא
פעולה אטומית ולכן מוגנת מפני גישה של יותר מתהליך אחד במקביל -
אז אם התהליך j המזהה שלו (j עצמו) קטן משלי - i, אז אני צריך לחכות.
בכל מקרה - מובטחת מניעה הדדית - משום שכל התהליכים נכנסים לפי תור - לפי
קבלת מספרם ואם יש מספר זהה - אז מבין אלו - זה עם המזהה הכי קטן יכנס
לפני האחרים עם אותו מספר - ולא ייתכן 2 תהליכים עם אותו מזהה.
לכן מובטחת לנו מניעה הדדית. לא ייתכן 2 תהליכים שימצאו בקטע הקריטי בו זמנית.

המשך...

בס"ד

ויש הבטחה גם להעדר הרעבון, משום שלא מתקיים התנאי ההכרחי של מעגל - שכן, התהליך שקיבל את המספר הכי קטן מבין אלו שממתינים להכנס - כאשר הקטע הקריטי פנוי (אם לא פנוי אז מחכה שיתפנה), הוא זה שיצא לקטע הקריטי, ואם יש כמה כאלו - האחד עם ה id הכי קטן מתוכם יוכל להכנס.

לכן בכל שלב, אם הקטע הקריטי פנוי - מישהו יוכל להכנס.

אם הקטע הקריטי תפוס - הוא כהגדרה יתפנה מתישהו ומי שממתינים יסמן את המספר שלו כ-0 וכך יוכל לתת למי שמחכה בתור להכנס, כי התנאי הראשון של הלולאה while לא מתקיים למי שמחכה לו.

ההמתנה היא מוגבלת, משום שתהליך שקיבל מספר - נכנס לתור ואף אחד לא יוכל לעקוף אותו.

אם יגיע תהליך חדש שירצה להכנס לקטע הקריטי - הוא כהגדרה יקבל מספר שגדול מכל התהליכים שכבר ממתינים - כי לוקח את max המספרים ועוד 1.

לכן לא יוכל לעקוף אף אחד.

ומשום שכל פעם מישהו אחר נכנס - לפי התור, ובסופו של דבר כל אחד שממתין יגיע תורו.

③

בס"ד

ב. ראו חלק שני של סעיף א'. שם עניתי על זה.
אינו שוויוני: שבכל אחד לא יכול לצעוק אבל אחד יכול תמיד בסעיפי
של דבר עקף ונצח.
מנצלים מספרים שמהווים מספר כמו כמו בבחירה מרקדת, אי אפשר לצעוק
ומי שהגיע ראשון נעשה ראשון לפני ומי שהגיע אחרון יקבל את המספר
שהוא יותר גבוה מכל מי שממתין לכן נעשה לסוף התור.

ג. לא, כמו הפרוטוקול אינו מבטיח מניעה הדדית.
נניח תהליך 5 מגיע, ולכשמגיע הוא היחיד לכן יקבל מספר 1,
נניח כי בזמן, תהליך 5 היחיד - לכן עבר את כל הלולאה ונכנס לקטע הקריטי ועכשיו שם.
כעת נניח הגיעו עוד 2 תהליכים 6,7 (אחד אחרי השני, לא יודע) וקיבלו
את המספרים 2 ; [illegible] בהתאמה
השני קיבל 0 בגלל ~~[illegible]~~ הבעיה הבעיה. ! 3·1:3=0
לכן, תהליך מספר 6 מחכה כי תהליך 5 עם מספר 1 ותהליך 6 עם מספר
2 ולכן תורו עדין
אבל תהליך 7, עם מספר 0, לכן בלולאה for לא יעבור בכלל את מתהליכים,
בטח לו בגלל שלא נכנסו לתור, ועבר לו אבל תהליך 5 או 6,
מאחר שמבצעי "צעקה" את התור, ולא מקיים את תנאי הלולאה הפנימית על מה
שממתין.
לכן גם הוא יצליח להיכנס לקטע הקריטי, אבל תהליך 5 עדיין שם!!
ואין מניעה הדדית.
לכן הפרוטוקול לא מבטיח מניעה הדדית.
המשך..

~~ג. לא, הבטיח את מניעה הדדית. משום שאם יש מישהו שממתין בלולאת while (השני) הוא נמצא לפני~~
~~אבל לא מגיע לזה שמחכה אינסוף בלולאת for ...~~
~~מובטח לנו שמי שמגיע אחרון יקבל מספר יותר גדול ממנו,~~
~~לכן, גם אם תהליך חדש שהגיע נמצא באמצע לולאת ה-for את המספר שלו, ולא~~
~~סיים, ולא [illegible] מספר כזה של הוא בודאות - מובטח שיקבל מספר גדול יותר,~~
~~לכן, בכל מקרה [illegible] התנאי השני של הלולאה while לא מתקיים~~
~~ואם 2 הגיעו יחדיו (או יותר מ2) אז אם הם קיבלו אותו מספר - בכל מקרה~~
~~מי שה-id שלו גדול יותר עובר ראשון כי כבר [illegible] שאינו הסעיף שהמתנה,~~
~~ואם [illegible] כך, אזי מתקיים המסקנה, שהמתנה~~

⑥

בע

לא, לא מושג מניעה הדדית.

נניח ש 2 תהליכים הגיעו בו זמנית לעדכון המספר שלהם

~~[illegible] שם [illegible] 1~~ (נניח אחד ראשון ושני, אבל לפני שהספיקו לעדכן השמה למספר)

כל אחד מהם חושב כשמקבל את ה max שהם קראו והוסיפו לו 1,

שניהם קיבלו תוצאה של 1,

אחרי שמתחילים [illegible] גם עדכנו את המספר, נקרא לתהליך זה עם id=6

והמשיך, [illegible] ללולאה - הוא עובר הכל, משום שגם התהליך השני לא הספיק

עוד לעדכן את המספר שלו. לכן גם השני המספר עדיין 0 אצלו.

לכן תהליך 6 נכנס לקטע הקריטי.

נניח כעת התהליך השני, נקרא לו תהליך 5, מעדכן השמה של המספר, 1 למספר

שלו, ומגיע ללולאה, כאשר עובר את כל האלו שקטנים לו לגבי מספר, 6 ≥ 5

[illegible] ומגיע להשוות j=6 ואז רואה שיש לו אותו מספר, אבל [illegible]

לכן גם הוא עובר ונכנס לקטע הקריטי.

ואין מניעה הדדית!

המשך...

בס״ד

שאלה 2.

בשלב ראשון ה MMU צריך לתרגם כתובת, הוא צריך לדעת לאיזה רמה לפנות, הפנימי או החיצוני.

דפים רגילים 4Kbyte כלומר 2^{12} bytes ⇐ 12 סיביות עבור ההיסט
דפים " 4Mbyte " 2^{22} bytes ⇐ 22 סיביות עבור ההיסט

כיוון שבחלק הפנימי ישנו גודל בלוק גדול יותר יהיו פחות בלוקים ופחות כתובות משום שהצטרכנו לחלק באופן שווה בין 2 הרמות הקודמות.
לכן גודל הטבלאות עצמן בשלמותה תהיה שווה בין 2 החלקים.
ההבדל יהיה בסיביות כמה סיביות ילכו להיסט וכמה ל frame.

ה MMU צריך אינדיקציה להבדיל בין ה כתובות וירטואליות לחלק הפנימי, כלומר של תהליך שקיבל הדפים הגדולים, לבין החלק החיצוני. על מנת לדעת כמה סיביות לקחת לאינדקס וכמה להיסט.

פתרון אחד הוא לתת לתהליכים שצריכים כתובות בחלק הגבוה מרחב וירטואלי שבו הסיביות המשמעותיות הראשון 1 — זה החלק העליון.
כלומר שהכתובות הפנימיות כל מה שמתחיל בסיביות השמאליות כ-0 זהו ~~החלק~~ החלק הפנימי יותר, וכל מה שמתחיל בסיביות השמאליות כ-1 זהו החלק החיצוני יותר,
לכן צריך כל התהליכים לתת כתובות וירטואליות שמתחילות ב 1
וה MMU יבדוק האם טבלת הדפים שלו - של התהליך מתחילה בכתובות של 1,
אבל יש פה בעיה יותר קשה:
כל תהליך יקבל כמו תמיד מרחב כתובות וירטואלי שמתחיל מ-0, אך ~~[illegible]~~
~~[illegible]~~ במרחב כתובות נוסף (אפשר יחד עם טבלת הדפים של
התהליך, אבל החישובים שצריך לעשות כי מאחר שמוסיפים שטבלת הדפים תהיה רגילה
מאוד ושלא [illegible] אלא עוד סיביות) [illegible] לא בגלל טבלת Hash
עבור כל תהליך שהם הוא בחלק הראשון -0 או הוא בחלק השני 1,

המשך...

כאשר מגיע תהליך - אז בטבלת הדפים שלו יש 2^{n-12} כניסות ~~[illegible]~~ כאשר n מציין את גודל הכתובת הוירטואלית, סימן שתהליך זה קטן יחסית.
בחלק הראשון הימני, ורק ה MMU עד את 12 הביטים הימניים להיסט והשאר n לאינדקס,
וזה יכול לעבוד טוב כי כניסות יש, כי שבודק את האינדקס ועושה כמו בטבלה של התהליך.
אם מספר הכניסות 2^{n-22} אז ~~[illegible]~~ לתהליך ~~[illegible]~~ הוא יתבצע כניסות בחלק השני,
ולכן יכול לקחת 22 ביטים להיסט ומשאר לאינדקס ובכך לחסוך את הכתובות.

ומכאן שאם תהליך ~~[illegible]~~ חדש מגיע ~~[illegible]~~ הוא מקבל או ~~[illegible]~~ טבלה בגודל 2^{n-12} כניסות או 2^{n-22} כניסות, בהתאם לצורך.

2

$2^{27}B \Leftarrow 128MB$
אם התהליך הוא עם דפים של 4KB זה אומר שגודל הטבלה שלו 2^{27-12} כלומר 2^{15}.
אם התהליך הוא עם דפים של 4MB זה אומר שגודל הטבלה שלו 2^{27-22} כלומר 2^{5}.
נתבונן בכתובות הבאות:
1. לפי ההגדרה בשאלה, בהינתן טבלת הדפים, מספר המסגרת הפיזית שווה למספר הדף + 1 (וזה מגיע גם שהכתובת חוקית, כלומר שלא חורגת מטבלת הדפים של התהליך - האינדקס קטן וכו'...)
במקרה זה, מהדף 1, לא ייתכן שמדובר בחצי העליון של הכתובות הפיזיות, משום שלא משנה איך נחלק את החלק של ההיסט, לפי 12 ביטים או 22 ביטים בכל מקרה כאשר נוסיף 1 למספר הדף - לאינדקס שנמצא בביטים השמאליים, נקבל כי הביט השמאלי ביותר הוא 0, ולא שייך לחצי העליון של הכתובות
אם $\underbrace{0000100.0000000000}_{\text{אינדקס}}.\underbrace{0000000011}_{\text{היסט}}$, לפי ההגדרה $\Leftarrow$ $\underbrace{000100.0000000|0000000001}_{\text{כתובת פיזית}}$

(7)

2. עבור שורה 2.

הסבר קצר לקודם - לא משנה כתובת שהוספתי השמאלי כיוון הוא 1 עד כתבי, למעלה. לכן מדובר בחצי הזיכרון של הכתובת הפיזית. (כי עד 1 עכשיו יש 1 בשמאלי...)

לכן התרגום:

$\underbrace{10001}_{\text{אינדקס}}\underbrace{00.0000000000.0000000100}_{\text{היסט}}$ ⇐ הכתובת הפיזית ⇐

$\underbrace{10010}\underbrace{00.0000000000.0000000100}_{\text{היסט}}$

↓

כי הגיוני שהאינדקס יהיה 10010 משום שהכתובת הפיזית היא 64 Mbytes
ויש פה משהו סביר כאן!

כשעברה 1 הפכתי את ה-0 השמאלי (זה היה אולי הגיוני בגלל שהספרה הבאה מימין היתה שונה מ 1.)

אבל סתם אי אפשר להשמיט את ה 1 המוביל. זה יפגע בכל המשמעות.

לכן בהאינדקס 10001 בכל הדפים של התהליך צריך להיות מספר מסגרת הגיוני!
ולא כמו שנראה.

כלומר צריך להיות שם משהו עם 4 ספרות ולא עם 5!

נניח 1010 יהיה סביר. מוליו ~~[illegible]~~ 4ר3 את ההיסט.

משל

⑧

בס"ד

שאלה 3:

ואופן הנעילה

א. המטרה העיקרית של רכיב ה DMA זהו תקשורת עם בקר התקן I/O
כך שיוכל להעביר נתונים ישירות מהזיכרון להתקן ומההתקן לזיכרון, כך
שה CPU יחדל להיות מעורב בכל תהליך העברה - והוא יוכל להתפנות לעיסוקיו האחרים.
ה CPU נדרש רק לבצע נעילה של אזורים מסוימים בזיכרון על מנת שתהליכים אחרים
ונוספים לא יגעו באזורים אלו, ולהורות את ה DMA מהיכן/לאיפה להעביר/
לקרוא נתונים. שחרור ה CPU מהצורך לבצע זו בעצמו משפר את היעילות.
ישנם 2 אפשרויות של נעילה - נעילה אחת, נעילה פר בלוקים.
(הייתי מפרט יותר על 2 שיטות הנעילה, אבל אין מספיק שטח בבחינה =(
הוא גם צריך לבטל את האפשרות של ה cache עבור אזורי זכרון אלו.

ב. הוו, כעת נראה שאפשר לדבר על 2 השיטות של הנעילה.
כפי שאמרנו, ה CPU צריך לנעול אזורים בזיכרון, דבר זה - כפי שאמרנו
יש 2 שיטות.
1. שיטה ראשונה זה לנעול את כל האזור המבוקש בפעם אחת, דבר זה משחרר
את המעבד לעיסוקיו כך שלא יצטרך לנעול שוב עוד אזורים,
מצד שני - אם האזור נעול - ה CPU יצטרך להמתין זמן רב יותר
עד שהאזור ~~יפתח~~. ישוחרר.
ואם נעלם בלוקים - אז אפשר שה CPU יגש לבלוקים המשוחררים, אך ידרשו
יותר נעילות - כלומר יותר פעמים שה DMA יצטרך לגנוב זמן CPU
בכל מקרה - זה יכול לגרום להאטה בביצוע פעולות חישוב - במקרה הראשון -
אם מחכים למידע שיגיע מהתקן ה I/O עצמו
ובמקרה השני - אם המעבד צריך להיות מעורב יותר פעמים בנעילת הבלוקים
ולכן פחות יכול לעסוק עוד בפעולות חישוביות במקביל לעבודת DMA.
דרכי התמודדות עקרוניות לבעיה נניח: שהתקן ה I/O יגש לאזורי זיכרון
ששייכים למערכת ההפעלה - כי לפי אלו בכל מקרה נעולים ויש בקרה של מערכת
ההפעלה על מי שניגש לאזורים אלו, דבר זה מאפשר ל CPU לא להצטרך
לנעול אזורים מיוחדים בזיכרון והוא יכול לעסוק בינתיים בפעולות חישוביות
שונות.

שמשון...

בס"ד

שאלה 4.

פונקציית ספריה היא פונקציה שעוטפת קריאת מערכת.
למשל בדוגמה הנתונה write זוהי קריאת מערכת אשר מקבלת File Descriptor, מצביע למחרוזת ואורך - כמות הבתים לכתיבה ומבצעת כתיבה ל fileDes שנתון.
ואילו printf זוהי פונקציה שמקבלת מצביע מחרוזת ואורך ומבצעת בפועל מימוש - ע"י זה שקוראת ל write קריאת מערכת.
לכל קריאת מערכת יש מספר קריאת מערכת ע"י מס' שמועבר ובעזרתו כאשר יתבצע trap אל הקוד קריאת המערכת, הקוד בהתאם למס' ידע מה צריך לבצע עבור הפעולה.

שאלה 5.

1. כן. ייתכן. נניח תהליך א' שעשה פעמיים pouch לייצר קונטיינר - את הראשון לו ואת הקונטיינר מקונן אליו בתוכו -
אז התהליך הראשון שנוצר בתוך הקונטיינר יהיה לו PID (1) כמו התהליך הראשון בקונטיינר השני שה PID שלו 1. גם הבנים שלהם ייתכן שיהיה להם מס' זהים בלוקלי. כי הקונטיינרים הם יחודיים.
2. לא ייתכן. משום שלינוקס משתמש למזהה חד חד ערכי ע"י מערכת ההפעלה ושימוש החלוקה שצריך להכיל את כל התהליכים. לכן לא ייתכן שיהיה PID גלובלים שווים.

שאלה 6.

(הנחה: [illegible]) unshare מבודד שהתהליך ~~[illegible]~~ שהתהליך הנוכחי מתנתק ונכנס למרחב ריקים משלו, אין זה אומר שהוא לא יוכל לגשת לקבצים גלובליים!
אם הוא שנמצא במרחב משלו ייצר קובץ בלי mount הקובץ הינו גלובלי וגם הוא של הכלל יוכל לגשת אליו. (קובץ, ספריה וכו'...)
אם היה נוצר תקיה ועשה mount לתקיה הזו ל device filesystem אחר, אז התקיה הזו היא ה mount point רק למי שנמצא במרחב זה, כלומר הוא שנמצא במרחב זה יכול, אך מי שהוא לפני ה unshare לא יוכל לגשת, עבורו כיוון כך את ה mount point הוא ראה שזו תקיה ריקה.
עכשיו הרכבה השנייה של התהליך שנוצר ב NS נפרד, גלובליים לכולם, אלא אם כן הוא עושה mount למרחב שלו.

⑩

בס"ד

חלק ג

7. ד (ד הוא נכון. אבל לדעתי גם עם ג אפשרי.)
8. ג
9. א
10. ג
11. ב
12. א